

X68000 31 kHzモードで 60 Hz垂直同期を生成・実測する

C実験からpx68kの修正、WebX68k・X68000 Zでの約60 Hz確認まで

垂直タイミングの基礎・CRTC R04～R07・V-DISP実測・エミュレータ比較

X68000の通常モードは約55 Hzで、MSXの60 Hzと一致しない

X68000（31 kHzの通常モード）

約55.5 Hz

1秒間の垂直同期回数

MSX（NTSCモード）

60 Hz

ゲーム進行の基準にも使われる

『X68000用MSXシミュレータ「MS.X」開発秘話』は、この差を最初の壁として説明しています。

「MSXはNTSCモードで60Hz。ところが、X68000の通常の画面モードは約55Hz。」

垂直同期を基準に動くソフトでは、この約4.5 Hzの差が動作速度やタイミングの差になります。

[1] 電子版 v1.0, PDF p.37（本文 p.35）。周波数の代表値は同 PDF p.8（本文 p.6）。

MS.XはCRTCを直接設定し、60 Hzの画面モードを作った

書籍に記された対応

「X68000のCRTCレジスタを直接いじって、垂直同期が60Hzになるように画面モードを作り変えることで対応しました。」

CRTCとは

CRT Controllerの略で、1画面を構成する水平・垂直の同期、空白期間、表示期間を決める回路です。

本資料で行う確認

1. R04～R07へ525ラインの値を設定
2. V-DISPの600周期を計測
3. 約10秒なら約60 Hzと判定

[1] 電子版 v1.0, PDF p.37（本文 p.35）：引用文とCRTCの括弧内説明。

水平タイミングを保ったまま、垂直同期を約60 Hzへ変更する

- 01 まずCRTCと垂直同期の関係を整理する
- 02 次にMS.Xを参考に、終了時の制約を明示したC実験を作る
- 03 最後にpx68kの修正とWebX68kでの確認結果を示す

設定した垂直同期を実測し、終了時は保存したIOCSモードを再設定する。

MS.Xの実装からエミュレータ改善案までを5段階でたどる

値やコードを見る前に、垂直タイミングとレジスタの意味を確認する。



前半で生成と測定を説明し、後半でエミュレータが同じ動作をどう再現するか調査する。

XEIJの実装ではR04～R07を垂直4区間の境界として扱う

R04

crtR04VFrontEndCurr

垂直フロントポーチ終了

R05

crtR05VSyncEndCurr

垂直帰線期間終了

R06

crtR06VBackEndCurr

垂直バックポーチ終了

R07

crtR07VDispEndCurr

垂直映像期間終了

各レジスタは区間の長さそのものではなく、境界となるラスタ番号を保持する。

[2] XEIJ / CRTIC.java : 垂直同期図、R04～R07のCurrent値とコメント。

MS.X公式ソースはR04～R07をCRTCの実アドレスへ割り当てる

```
#define CRTR_04 (*(volatile uint16_t*)0xe80008)
#define CRTR_05 (*(volatile uint16_t*)0xe8000a)
#define CRTR_06 (*(volatile uint16_t*)0xe8000c)
#define CRTR_07 (*(volatile uint16_t*)0xe8000e)
```

src/vdp/ms_vdp.h L33-L36

この引用から分かること

1. MS.X自身がX68000のCRTCを直接操作する
2. 対象はR04～R07の4レジスタ
3. crtc60hz.cも同じアドレス体系を利用する

[3] MS.X公式リポジトリ / src/vdp/ms_vdp.h, L33-L36。

31.5 kHzのまま525ラインにすると60 Hzになる

$$31,500 \text{ lines/s} \div 525 \text{ lines/frame} = 60.00 \text{ frames/s}$$

水平周波数

31.5 kHz

1ラインの時間は変更しない

垂直総ライン数

525 lines

4区間の合計

垂直周波数

60.00 Hz

1秒あたりのフレーム数

垂直周波数は、水平ライン周期と1フレームの総ライン数から決まる。

4区間の長さを累積してR04～R07の値を求める

VSYNC 2、バックポーチ33、表示480、フロントポーチ10を順に配置する。

VSYNC	バックポーチ	表示領域	フロントポーチ
2	33	480	10

$$2 + 33 + 480 + 10 = 525 \text{ lines}$$

$$R05=1 \rightarrow R06=34 \rightarrow R07=514 \rightarrow R04=524$$

各レジスタには区間の最終位置を格納するため、総数より1小さい値になる。

MS.Xの設定をC版のR04～R07へ書き込む

垂直CRTC設定

```
R04 = 0x020C; /* 525 - 1 */
R05 = 0x0001; /* VSYNC end */
R06 = 0x0022; /* display start */
R07 = 0x0202; /* display end */
```

MS.X / crt60hz.c

MS.Xの垂直タイミング設定を参考に、C版でも525ラインの周期を作る。

1

標準モードから開始

IOCS mode 12で既知の31 kHz系水平タイミングを作る。

2

垂直値だけ変更

R04～R07へ導出した4つの値を書き込む。

3

生成後に実測

V-DISPを数え、設定結果を実時間で確認する。

MS.Xを設定値の根拠とし、C版は生成された垂直同期を測定する。

CRTCレジスタの多くは書き込み専用で、現在値を読み戻せない

X上で「R04～R07は読み出すと0が返る」とご指摘をいただき、実装と資料を修正した。

情報	読み出し	サンプルでの扱い
IOCS CRT mode番号	可能	_iocs_crtmod(-1)で保存
CRTC R20・R21	可能	今回は直接保存しない
CRTC R04～R07	不可（0が返る）	読み戻さない
独自CRTCタイミング	完全取得不可	完全復元は保証しない

R04～R07の「保存」ではなく、IOCSモードの「再設定」と表現する。

指摘を受け、R04～R07の読み戻し処理を削除した

修正前（誤り）

```
for (i = 0; i < 4; ++i) {  
    saved_crtc[i] =  
        read_crtc(crtc_regs[i]);  
}
```

実機ではR04～R07の読み出し結果は0

修正後

```
old_mode = _iocs_crtmod(-1);  
_iocs_crtmod(12);  
/* write and measure R04-R07 */  
_iocs_crtmod(old_mode);
```

保存するのはIOCSモード番号だけ

終了時は保存したモードをIOCSへ渡し、そのモードの標準CRTC設定を再設定させる。

IOCSモードを保存し、既知のmode 12から垂直値だけを変更する

修正後の要点

```
old_mode = _iocs_crtmod(-1);
_iocs_crtmod(12);
write_crtc(R05, 0x0001);
write_crtc(R06, 0x0022);
write_crtc(R07, 0x0202);
write_crtc(R04, 0x020c);
/* after measurement */
_iocs_crtmod(old_mode);
```

1

保存するのはモード番号

2

R04～R07は読み戻さない

3

終了時はIOCS標準設定へ再設定

実行前に他のアプリケーションが独自設定したR04～R07は完全復元できない。

GPIP bit 4を監視して次のV-DISP立ち上がりを待つ

現在の信号状態に関係なく、次の開始点を1回だけ検出する。

1

GPIP bit 4

General Purpose Input/Output Port。
\$10はV-DISP (0: 空白、1: 表示)。\$40はCIRQ。

2

一度Lowを待つ

呼び出し時がHighでも同じフレームを数えない。

3

タイムアウト

信号が変化しない場合は復元処理へ進む。

```
WAIT_VDISP()
```

```
#define GPIP_VDISP 0x10
```

```
while ((read_gpip() & GPIP_VDISP) != 0) { }
```

```
while ((read_gpip() & GPIP_VDISP) == 0) { }
```

```
crtc60hz.c
```

1回のwait_vdisp()が、次のV-DISP開始を1回数える。

600回目のV-DISP直後に終了時刻を取得する

測定区間

```
wait_vdisp();
start = _iocs_ontime();

for (frame = 1; frame <= 600; ++frame) {
    wait_vdisp();
    if (frame == 600) end = _iocs_ontime();
}
```

crtc60hz.c

開始点と終了点をV-DISPの直後にそろえる。

1

開始

最初のV-DISP直後にstartを取得する。

2

600区間

その後のV-DISPを600回待つ。

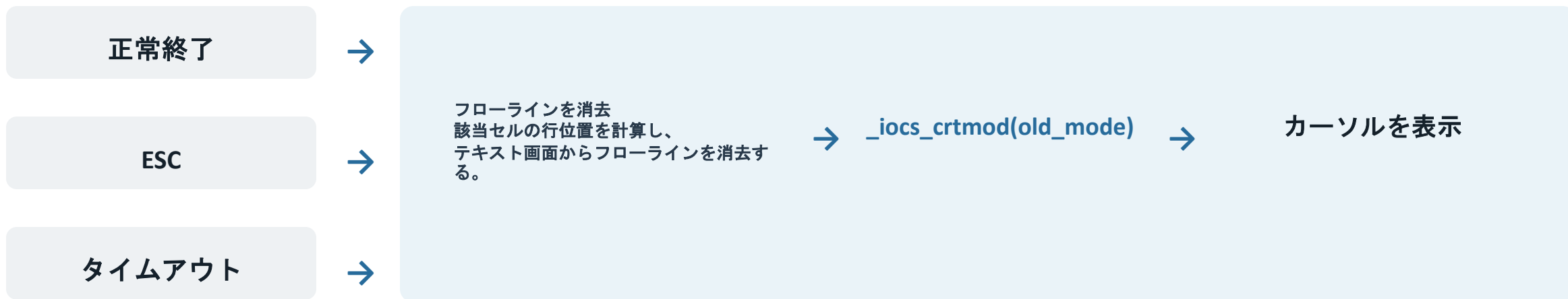
3

終了

描画処理より前にendを取得する。

$$\text{Hz} \times 100 = 600 \times 10000 \div \text{経過時間 (1/100秒)}$$

どの終了経路でも、保存したIOCSモードを再設定する



「元のCRTC値を完全復元」ではなく「元のIOCSモードの標準設定へ戻す」

実機・エミュレータ・X68000 Zで約60 Hz動作を確認

実機	XEiJ	XM6 TypeG	WebX68k	MPX68K
約60 Hz	約60 Hz	約60 Hz	約60 Hz	60.00 Hz
525ライン設定とIOCSモード再設定を確認	600 V-DISPが約10秒	600 V-DISPが約10秒	px68k更新後、crtc60hz.xで確認	最新版でCRTC由来の周期計算を確認

追加確認: X68000 Zでもcrtc60hz.xにより60.0 Hz動作を確認した。

CRTC値から実時間まで4段階を一貫した周期でつなぐ

WebX68k対応後も、正しいタイミング実装を理解するための基本経路になる。

1

CRTC設定

R04～R07から総ライン数と表示区間を求める

2

ラストイベント

1ライン時間に従ってラスト番号を進める

3

MFP GPIF

表示開始・終了位置でbit 4を変化させる

4

実時間同期

CRTC由来の周期をCPU・タイマー・音声・AV通知で共有する

XEiJはラスト番号に応じてV-DISPの開始・終了を通知する

```
// CRTC.java L2878-L2882
if (n != crtVDispStart) {
    TickerQueue.tkqAdd (crtTicker = DuplicationOmitIdleSync,
                        crtClock += crtHFrontLength);
} else {
    MC68901.mfpVdispRise ();
}

// CRTC.java L2979-L2982
if (n != crtVIdleStart) {
    TickerQueue.tkqAdd (crtTicker = DuplicationOmitDispEvenSync,
                        crtClock += crtHFrontLength);
} else {
    MC68901.mfpVdispFall ();
}
```

意味

crtVDispStartでV-DISPを立ち上げ、
crtVIdleStartで立ち下げる。

R04～R07から得た境界を、
MFPの信号変化と次イベントの時刻へ反映している。

[2] XEiJ / CRTC.java L2878-L2882、L2979-L2982（公式公開ソース）。

XM6 2.06はR04～R07から垂直タイミングを再計算する

```
crtc.v_sync = ((p[4] & 0x3ff) + 1);  
crtc.v_pulse = ((p[5] & 0x3ff) + 1);  
crtc.v_back = ((p[6] & 0x3ff) + 1) - crtc.v_pulse;  
crtc.v_front = crtc.v_sync - ((p[7] & 0x3ff) + 1);
```

vm/crtc.cpp L782-L785 / CRTC::ReCalc()

公開ソースで確認できること

R04を全ラスタ数、R05～R07を各境界として扱い、垂直同期・バックポーチ・フロントポーチを再計算している。

引用対象はXM6 TypeGではなく、公開されているXM6 2.06のソース

。

[4] XM6 2.06公開ソース / vm/crtc.cpp L782-L785。

修正前はCRTC由来の周期がWebX68kの全処理へ届かなかった

修正前の不整合

CRTC R00 / R04 / R20



派生タイミング



CPU・タイマー・音声・AV通知

1

周期の分断

CRTCを書き換えても、固定周期を参照する処理が残っていた。

2

経路ごとの差

CPU、ラスタ、タイマー、音声、フレーム通知が同じ周期を共有していなかった。

3

観測された結果

crtc60hz.xの525ライン設定がWebX68kの約60 Hz動作へ反映されなかった。

問題は画面表示だけでなく、CRTC周期を参照する複数の時間経路にまたがっていた。

px68kでCRTC周期を再計算し、すべての時間経路へ共有した

マージされた処理の要点

```
R00 + R04 + R20
  ↓
CRTC-derived timing
  ↓
CPU / raster / timer
AV info / audio / frameskip
```

1

CRTC設定から計算

水平・垂直タイミングから現在のフレーム周期を求める。

2

全経路で共有

CPU、ラスタ、実時間タイマー、AV通知、音声、フレームスキップが同じ周期を参照する。

3

変更と読込に追従

レジスタ書き込み時に更新し、旧セーブステート読込時も派生値を再計算する。

PR #1のマージ後、WebX68kでcrtc60hz.xが約60 Hzで動作することを確認した。

px68kはCRTC設定から求めた周期を実行系全体で共有する

x68k/crtc.c L255-L266

```
uint32_t CRTC_GetFrameClocks(void)
{
    uint32_t oscillator, divider, columns;
    uint64_t clocks;

    if (VLINE_TOTAL != 0 &&
        CRTC_GetHorizontalTiming(&oscillator, &divider, &columns))
    {
        const uint64_t numerator = CRTC_TIMING_CLOCK * divider * 8U *
            columns * VLINE_TOTAL;
        clocks = (numerator + oscillator / 2U) / oscillator;
    }
}
```

1

コアで計算

R00、R04、R20から水平・垂直タイミングを求める。

2

同じ周期を参照

CPU、ラスタ、タイマー、AV通知、音声、フレームスキップへ渡す。

3

異常値を防御

64 bit計算を使用し、範囲制限は同ファイルL274-L282で行う。

固定fpsではなく、現在のCRTC設定から求めた周期を唯一の基準にする。

31 kHzモードで約60 Hzを生成・実測し、 X68000 Zを含む複数環境で動作確認を完了した

1

映像信号の4区間とR04～R07の役割から、525ラインの設定値を導出した。

2

R04～R07は読み戻さず、終了時は保存したIOCSモードの標準設定へ戻す。

3

WebX68k、MPX68K、X68000 ZでCRTC設定に応じた約60 Hzを確認した。

謝辞

本検証は、XEiJ、MS.X、XM6の公開情報と実装から多くの知見を得て進めました。

R04～R07の読み出し制限に関するX上のご指摘と、WebX68k開発者によるレビュー、ステート互換性の確認、修正のマージに感謝します。

また、MPX68KでCRTC由来の周期計算を実装し、crtc60hzで60.00 Hzを確認してくださった開発者に感謝します。

いただいたご指摘により、実装と資料をより正確な内容に改めました。

参考資料

本文の引用番号と対応する一次資料・公開実装を一段組で掲載

[1] く に ち こ 『X68000用MSXシミュレータ「MS.X」開発秘話』電子版 v1.0 (2026-08-16)
<https://booth.pm/ja/items/8672923>

[2] XEIJ / CRTC.java - CRTCレジスタ定義とラスタ時刻処理
<https://stdkmd.net/xeij/source/xeij-CRTC.java.htm>

[3] MS.X公式ソース / src/vdp/ms_vdp.h
https://github.com/kunichiko/X68000_MSX_Simulator/blob/main/src/vdp/ms_vdp.h#L33-L36

[4] XM6 2.06 公開ソース / vm/crtc.cpp
http://retropc.net/pi/xm6/xm6_206s.zip

[5] px68k-libretro PR #1 - CRTC timing fix
<https://github.com/uraraworks/px68k-libretro/pull/1>

[6] MPX68K PR #26 - CRTC timing / 60.00 Hz verification
<https://github.com/YosAwed/MPX68K/pull/26>

[7] crtc60hz - 本資料で説明するサンプルの公開ソース
<https://github.com/renatus-novus-x/crtc60hz>

参照URLは2026-08-19時点。書籍の引用ページはPDFページと本文ページを併記した。